

硬體描述語言與邏輯合成實驗作業 - HW11

學號	姓名	檔名檢查	PDF 檔檢查
01153147	邱哲暘	(要自己檢查)	(要自己檢查)

* 主程式碼、測試程式碼皆需要截圖到作業上，波型圖需截圖

* 需用 pdf 檔繳交

* 請務必按照題目之設定撰寫程式

* 所有題目的 FSM 狀態皆須顯示文字

Q1.matrix_det_pipeline (矩陣乘法)

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} ab_{11} & ab_{12} & ab_{13} \\ ab_{21} & ab_{22} & ab_{23} \\ ab_{31} & ab_{32} & ab_{33} \end{bmatrix}$$

$$\det(AB) = \begin{vmatrix} ab_{11} & ab_{12} & ab_{13} \\ ab_{21} & ab_{22} & ab_{23} \\ ab_{31} & ab_{32} & ab_{33} \end{vmatrix}$$

輸入矩陣內元素 $a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{23}, a_{31}, a_{32}, a_{33}$

及 $b_{11}, b_{12}, b_{13}, b_{21}, b_{22}, b_{23}, b_{31}, b_{32}, b_{33}$ 為 4bits 的有號數。

計算後的結果 $ab_{11}, ab_{12}, ab_{13}, ab_{21}, ab_{22}, ab_{23}, ab_{31}, ab_{32}, ab_{33}$

為 11bits 的有號數。

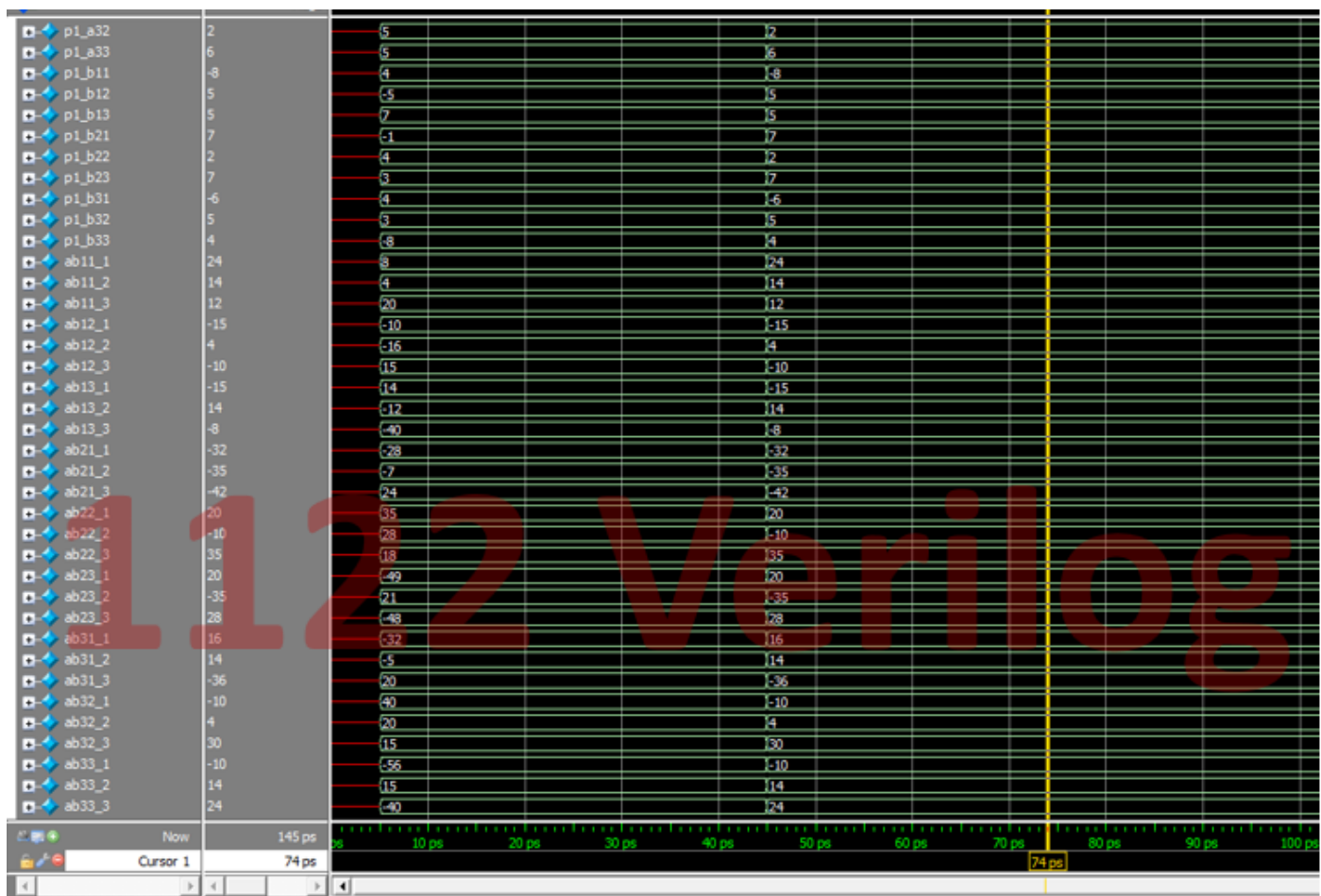
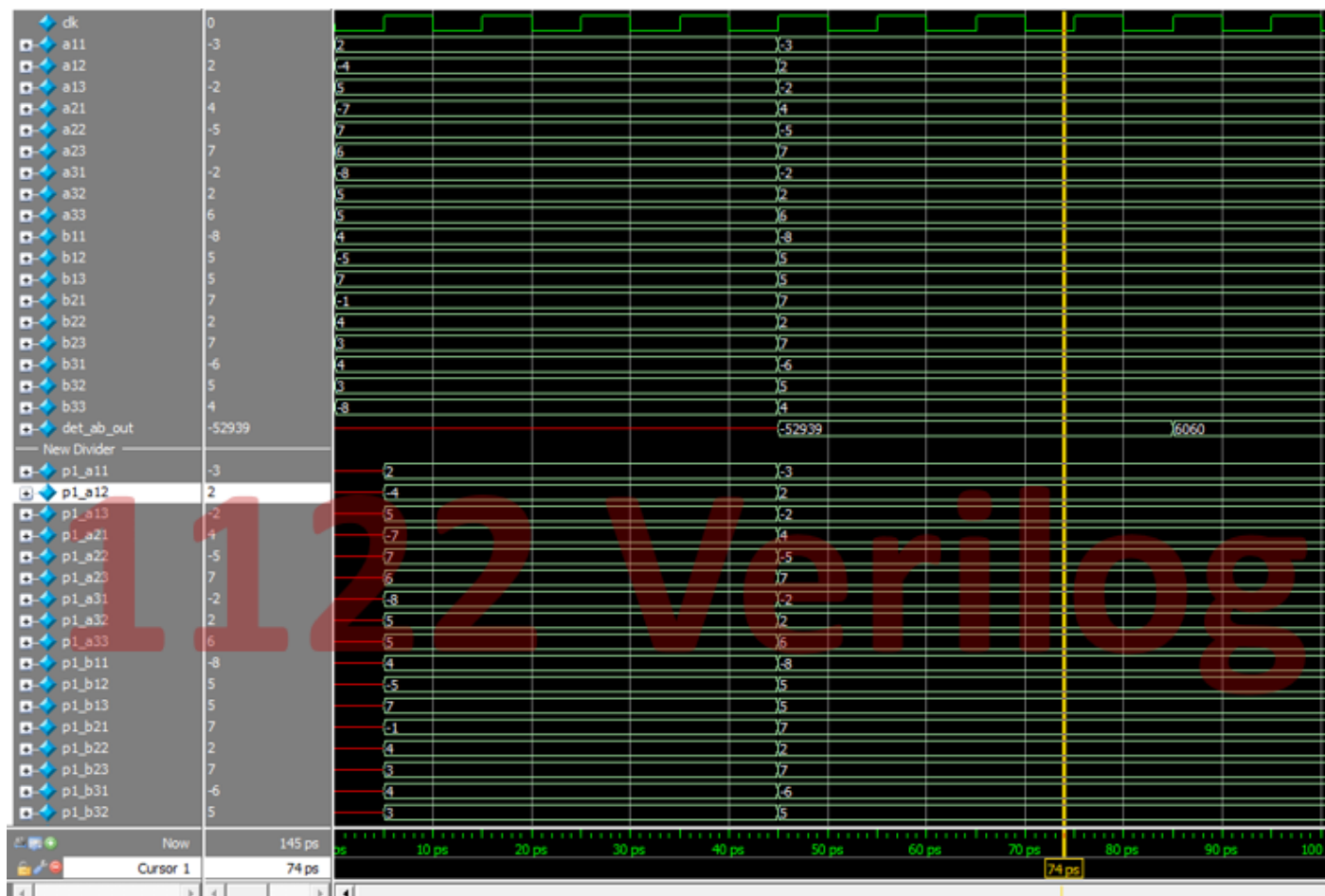
輸出結果 det_ab_out 為 32bits 輸出。

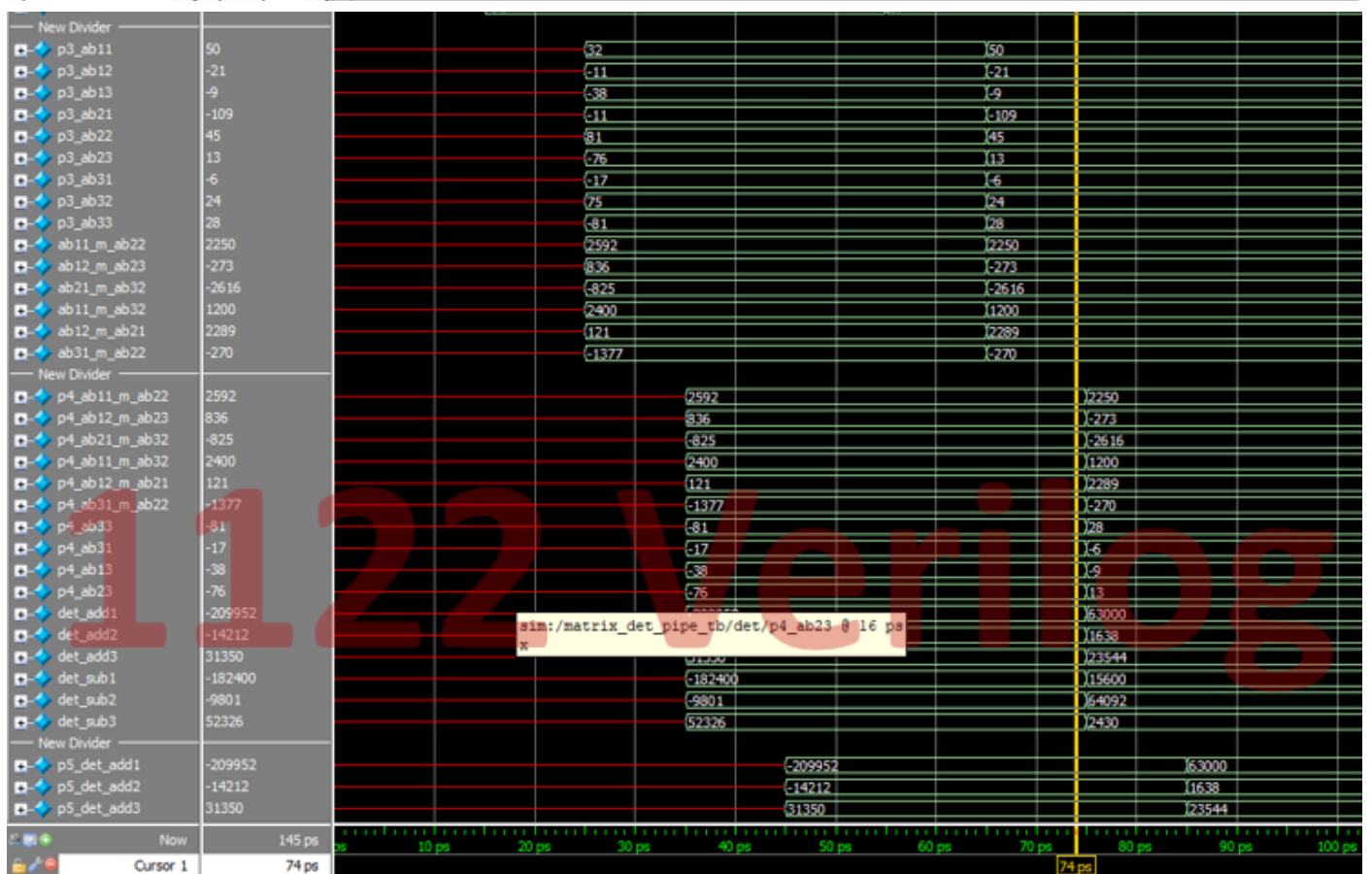
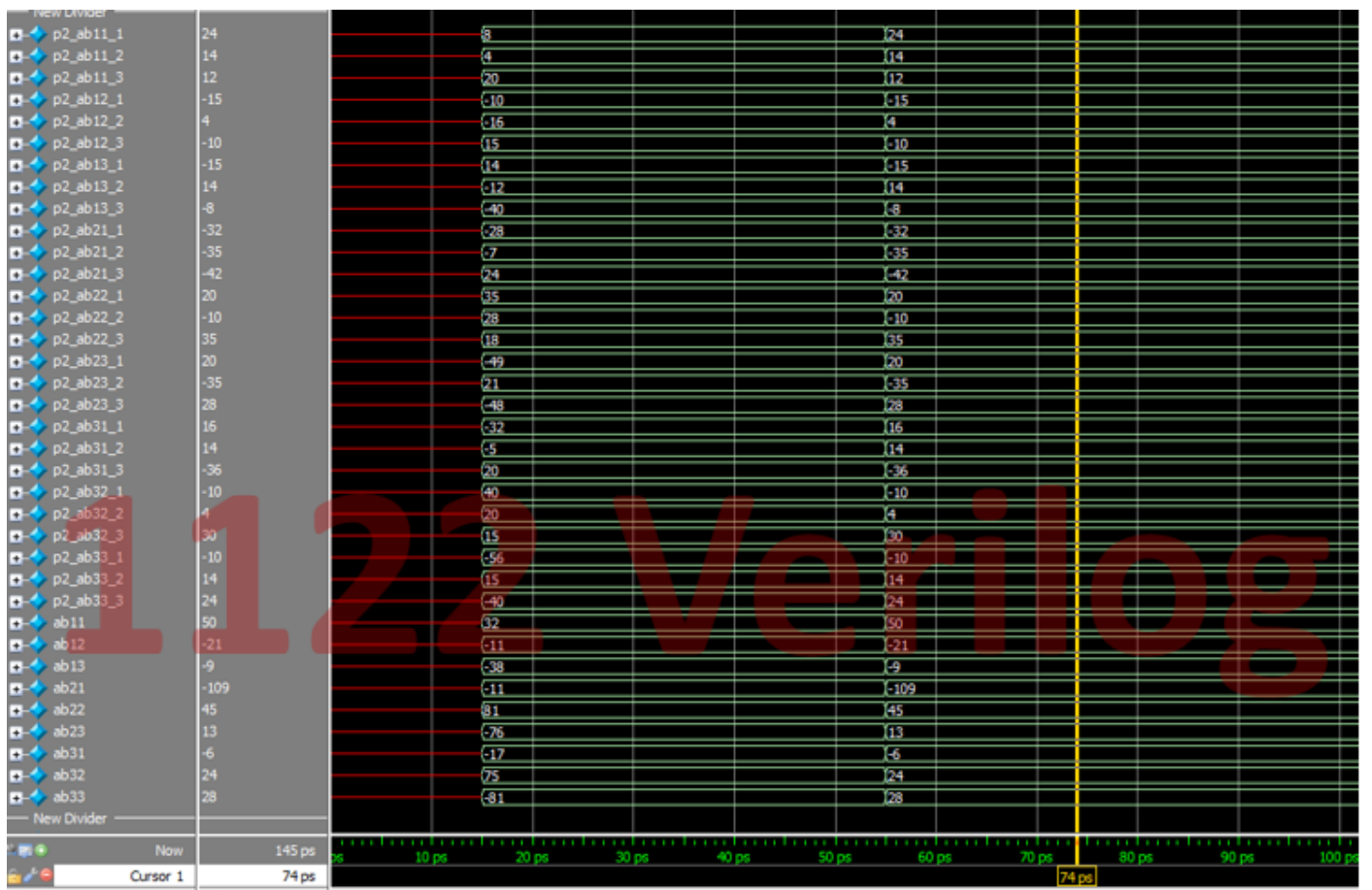
請在乘法器前與乘法器後加上 D_FF 做 PIPELINE 。

請依據 testbench 的程式片段撰寫測試程式 (Fig. 1-1)，波型圖如下 (Fig. 1-2):

<pre> always #5 clk = ~clk; initial begin clk = 0; a11 = 4'b0010; a12 = 4'b1100; a13 = 4'b0101; a21 = 4'b1001; a22 = 4'b0111; a23 = 4'b0110; a31 = 4'b1000; a32 = 4'b0101; a33 = 4'b0101; b11 = 4'b0100; b12 = 4'b1011; b13 = 4'b0111; b21 = 4'b1111; b22 = 4'b0100; b23 = 4'b0011; b31 = 4'b0100; b32 = 4'b0011; b33 = 4'b1000; </pre>	<pre> #45 a11 = 4'b1101; a12 = 4'b0010; a13 = 4'b1110; a21 = 4'b0100; a22 = 4'b1011; a23 = 4'b0111; a31 = 4'b1110; a32 = 4'b0010; a33 = 4'b0110; b11 = 4'b1000; b12 = 4'b0101; b13 = 4'b0101; b21 = 4'b0111; b22 = 4'b0010; b23 = 4'b0111; b31 = 4'b1010; b32 = 4'b0101; b33 = 4'b0100; #100; \$stop; </pre>
--	--

Fig. 1-1 testbench 的程式片段





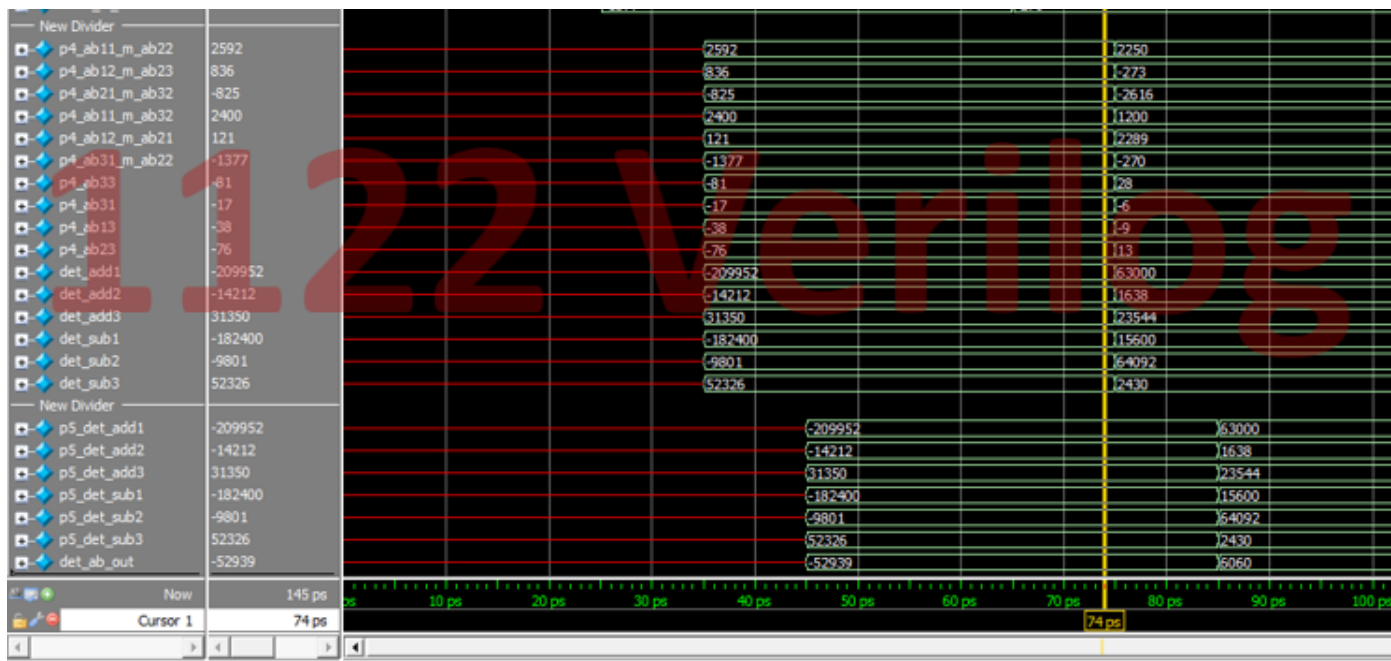


Fig. 1-2 波型圖

matrix_det_pipeline.v

主程式碼截圖：

//放主程式碼截圖

```

1 module matrix_det_pipeline(
2     input clk,
3     input signed [3:0] a11,a12,a13,a21,a22,a23,a31,a32,a33,
4     input signed [3:0] b11,b12,b13,b21,b22,b23,b31,b32,b33,
5     output signed [31:0] out_ab_det
6 );
7     reg signed [31:0] p1_a11,p1_a12,p1_a13,p1_a21,p1_a22,p1_a23,p1_a31,p1_a32,p1_a33;
8     reg signed [31:0] p1_b11,p1_b12,p1_b13,p1_b21,p1_b22,p1_b23,p1_b31,p1_b32,p1_b33;
9     wire signed [31:0] ab11_1,ab11_2,ab11_3,ab12_1,ab12_2,ab12_3,ab13_1,ab13_2,ab13_3;
10    wire signed [31:0] ab21_1,ab21_2,ab21_3,ab22_1,ab22_2,ab22_3,ab23_1,ab23_2,ab23_3;
11    wire signed [31:0] ab31_1,ab31_2,ab31_3,ab32_1,ab32_2,ab32_3,ab33_1,ab33_2,ab33_3;
12    reg signed [31:0] p2_ab11_1,p2_ab11_2,p2_ab11_3,p2_ab12_1,p2_ab12_2,p2_ab12_3,p2_ab13_1,p2_ab13_2,p2_ab13_3;
13    reg signed [31:0] p2_ab21_1,p2_ab21_2,p2_ab21_3,p2_ab22_1,p2_ab22_2,p2_ab22_3,p2_ab23_1,p2_ab23_2,p2_ab23_3;
14    reg signed [31:0] p2_ab31_1,p2_ab31_2,p2_ab31_3,p2_ab32_1,p2_ab32_2,p2_ab32_3,p2_ab33_1,p2_ab33_2,p2_ab33_3;
15    wire signed [31:0] ab11,ab12,ab13,ab21,ab22,ab23,ab31,ab32,ab33;
16    reg signed [31:0] p3_ab11,p3_ab12,p3_ab13,p3_ab21,p3_ab22,p3_ab23,p3_ab31,p3_ab32,p3_ab33;
17    wire signed [31:0] ab11_m_ab22,ab12_m_ab23,ab13_m_ab21;
18    wire signed [31:0] ab11_m_ab23,ab12_m_ab21,ab13_m_ab22;
19    reg signed [31:0] p4_ab11_m_ab22,p4_ab12_m_ab23,p4_ab13_m_ab21,p4_ab11_m_ab23,p4_ab12_m_ab21,p4_ab13_m_ab22;
20    reg signed [31:0] p4_ab31, p4_ab32, p4_ab33;
21    wire signed [31:0] det_add1,det_add2,det_add3,det_sub1,det_sub2,det_sub3;
22    reg signed [31:0] p5_det_add1,p5_det_add2,p5_det_add3,p5_det_sub1,p5_det_sub2,p5_det_sub3;
23    always @(posedge clk)//pipeline1
24    begin
25        p1_a11 <= a11;
26        p1_a12 <= a12;
27        p1_a13 <= a13;
28        p1_a21 <= a21;
29        p1_a22 <= a22;
30        p1_a23 <= a23;
31        p1_a31 <= a31;
32        p1_a32 <= a32;
33        p1_a33 <= a33;
34
35        p1_b11 <= b11;
36        p1_b12 <= b12;
37        p1_b13 <= b13;
38        p1_b21 <= b21;
39        p1_b22 <= b22;
40        p1_b23 <= b23;
41        p1_b31 <= b31;
42        p1_b32 <= b32;

```

```

43     p1_b33 <= b33;
44 end
45
46
47 assign ab11_1 = p1_a11*p1_b11, ab11_2 = p1_a12*p1_b21, ab11_3 = p1_a13*p1_b31;
48 assign ab12_1 = p1_a11*p1_b12, ab12_2 = p1_a12*p1_b22, ab12_3 = p1_a13*p1_b32;
49 assign ab13_1 = p1_a11*p1_b13, ab13_2 = p1_a12*p1_b23, ab13_3 = p1_a13*p1_b33;
50
51 assign ab21_1 = p1_a21*p1_b11, ab21_2 = p1_a22*p1_b21, ab21_3 = p1_a23*p1_b31;
52 assign ab22_1 = p1_a21*p1_b12, ab22_2 = p1_a22*p1_b22, ab22_3 = p1_a23*p1_b32;
53 assign ab23_1 = p1_a21*p1_b13, ab23_2 = p1_a22*p1_b23, ab23_3 = p1_a23*p1_b33;
54
55 assign ab31_1 = p1_a31*p1_b11, ab31_2 = p1_a32*p1_b21, ab31_3 = p1_a33*p1_b31;
56 assign ab32_1 = p1_a31*p1_b12, ab32_2 = p1_a32*p1_b22, ab32_3 = p1_a33*p1_b32;
57 assign ab33_1 = p1_a31*p1_b13, ab33_2 = p1_a32*p1_b23, ab33_3 = p1_a33*p1_b33;
58
59
60 always @(posedge clk) //pipeline2
61 begin
62     p2_ab11_1 <= ab11_1; p2_ab11_2 <= ab11_2; p2_ab11_3 <= ab11_3;
63     p2_ab12_1 <= ab12_1; p2_ab12_2 <= ab12_2; p2_ab12_3 <= ab12_3;
64     p2_ab13_1 <= ab13_1; p2_ab13_2 <= ab13_2; p2_ab13_3 <= ab13_3;
65
66     p2_ab21_1 <= ab21_1; p2_ab21_2 <= ab21_2; p2_ab21_3 <= ab21_3;
67     p2_ab22_1 <= ab22_1; p2_ab22_2 <= ab22_2; p2_ab22_3 <= ab22_3;
68     p2_ab23_1 <= ab23_1; p2_ab23_2 <= ab23_2; p2_ab23_3 <= ab23_3;
69
70     p2_ab31_1 <= ab31_1; p2_ab31_2 <= ab31_2; p2_ab31_3 <= ab31_3;
71     p2_ab32_1 <= ab32_1; p2_ab32_2 <= ab32_2; p2_ab32_3 <= ab32_3;
72     p2_ab33_1 <= ab33_1; p2_ab33_2 <= ab33_2; p2_ab33_3 <= ab33_3;
73 end
74
75
76 assign ab11 = p2_ab11_1 + p2_ab11_2 + p2_ab11_3;
77 assign ab12 = p2_ab12_1 + p2_ab12_2 + p2_ab12_3;
78 assign ab13 = p2_ab13_1 + p2_ab13_2 + p2_ab13_3;
79 assign ab21 = p2_ab21_1 + p2_ab21_2 + p2_ab21_3;
80 assign ab22 = p2_ab22_1 + p2_ab22_2 + p2_ab22_3;
81 assign ab23 = p2_ab23_1 + p2_ab23_2 + p2_ab23_3;
82 assign ab31 = p2_ab31_1 + p2_ab31_2 + p2_ab31_3;
83 assign ab32 = p2_ab32_1 + p2_ab32_2 + p2_ab32_3;
84 assign ab33 = p2_ab33_1 + p2_ab33_2 + p2_ab33_3;

```

```

85
86     always @(posedge clk) // pipeline3
87     begin
88         p3_ab11 <= ab11; p3_ab12 <= ab12; p3_ab13 <= ab13;
89         p3_ab21 <= ab21; p3_ab22 <= ab22; p3_ab23 <= ab23;
90         p3_ab31 <= ab31; p3_ab32 <= ab32; p3_ab33 <= ab33;
91     end
92
93     assign ab11_m_ab22 = p3_ab11 * p3_ab22;
94     assign ab12_m_ab23 = p3_ab12 * p3_ab23;
95     assign ab13_m_ab21 = p3_ab13 * p3_ab21;
96
97     assign ab13_m_ab22 = p3_ab13 * p3_ab22;
98     assign ab12_m_ab21 = p3_ab12 * p3_ab21;
99     assign ab11_m_ab23 = p3_ab11 * p3_ab23;
100
101     always @(posedge clk) // pipeline4
102     begin
103         p4_ab11_m_ab22 <= ab11_m_ab22; p4_ab12_m_ab23 <= ab12_m_ab23; p4_ab13_m_ab21 <= ab13_m_ab21;
104         p4_ab13_m_ab22 <= ab13_m_ab22; p4_ab12_m_ab21 <= ab12_m_ab21; p4_ab11_m_ab23 <= ab11_m_ab23;
105         p4_ab31 <= p3_ab31; p4_ab32 <= p3_ab32; p4_ab33 <= p3_ab33;
106     end
107
108     assign det_add1 = p4_ab11_m_ab22 * p4_ab33;
109     assign det_add2 = p4_ab12_m_ab23 * p4_ab31;
110     assign det_add3 = p4_ab13_m_ab21 * p4_ab32;
111
112     assign det_sub1 = p4_ab13_m_ab22 * p4_ab31;
113     assign det_sub2 = p4_ab12_m_ab21 * p4_ab33;
114     assign det_sub3 = p4_ab11_m_ab23 * p4_ab32;
115
116     always @(posedge clk)
117     begin
118         p5_det_add1 <= det_add1;
119         p5_det_add2 <= det_add2;
120         p5_det_add3 <= det_add3;
121         p5_det_sub1 <= det_sub1;
122         p5_det_sub2 <= det_sub2;
123         p5_det_sub3 <= det_sub3;
124     end
125
126     assign out_ab_det = p5_det_add1 + p5_det_add2 + p5_det_add3

```

```

127         - p5_det_sub1 - p5_det_sub2 - p5_det_sub3;
128
129     endmodule

```

matrix_det_pipeline_test.v

測試程式碼截圖：

//放測試程式碼截圖

```

131 module matrix_det_pipeline_test;
132 reg clk;
133 reg signed [3:0] a11,a12,a13,a21,a22,a23,a31,a32,a33;
134 reg signed [3:0] b11,b12,b13,b21,b22,b23,b31,b32,b33;
135 wire signed [31:0] out_ab_det;
136
137 matrix_det_pipeline T1(
138     .clk(clk),
139     .a11(a11),
140     .a12(a12),
141     .a13(a13),
142     .a21(a21),
143     .a22(a22),
144     .a23(a23),
145     .a31(a31),
146     .a32(a32),
147     .a33(a33),
148     .b11(b11),
149     .b12(b12),
150     .b13(b13),
151     .b21(b21),
152     .b22(b22),
153     .b23(b23),
154     .b31(b31),
155     .b32(b32),
156     .b33(b33),
157     .out_ab_det(out_ab_det)
158 );
159
160 always #5 clk = ~clk;
161
162 initial begin
163     clk = 0;
164
165     a11 = 4'b0010;
166     a12 = 4'b1100;
167     a13 = 4'b0101;
168     a21 = 4'b1001;
169     a22 = 4'b0111;
170     a23 = 4'b0110;
171     a31 = 4'b1000;
172     a32 = 4'b0101;

```

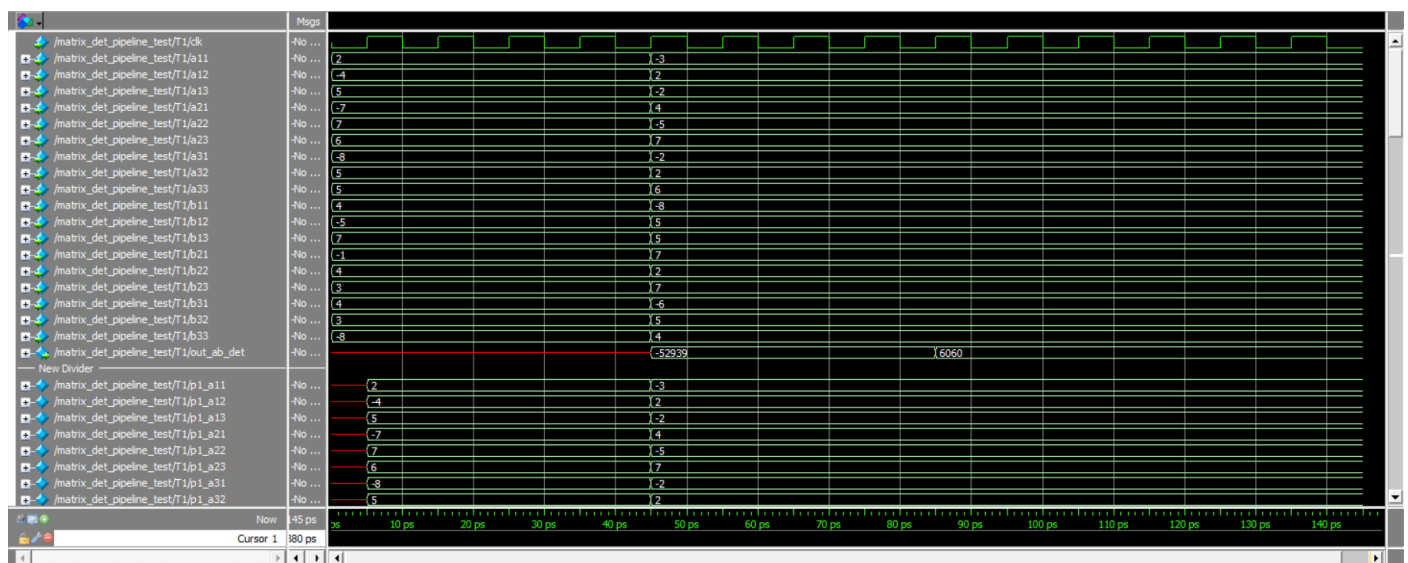
```

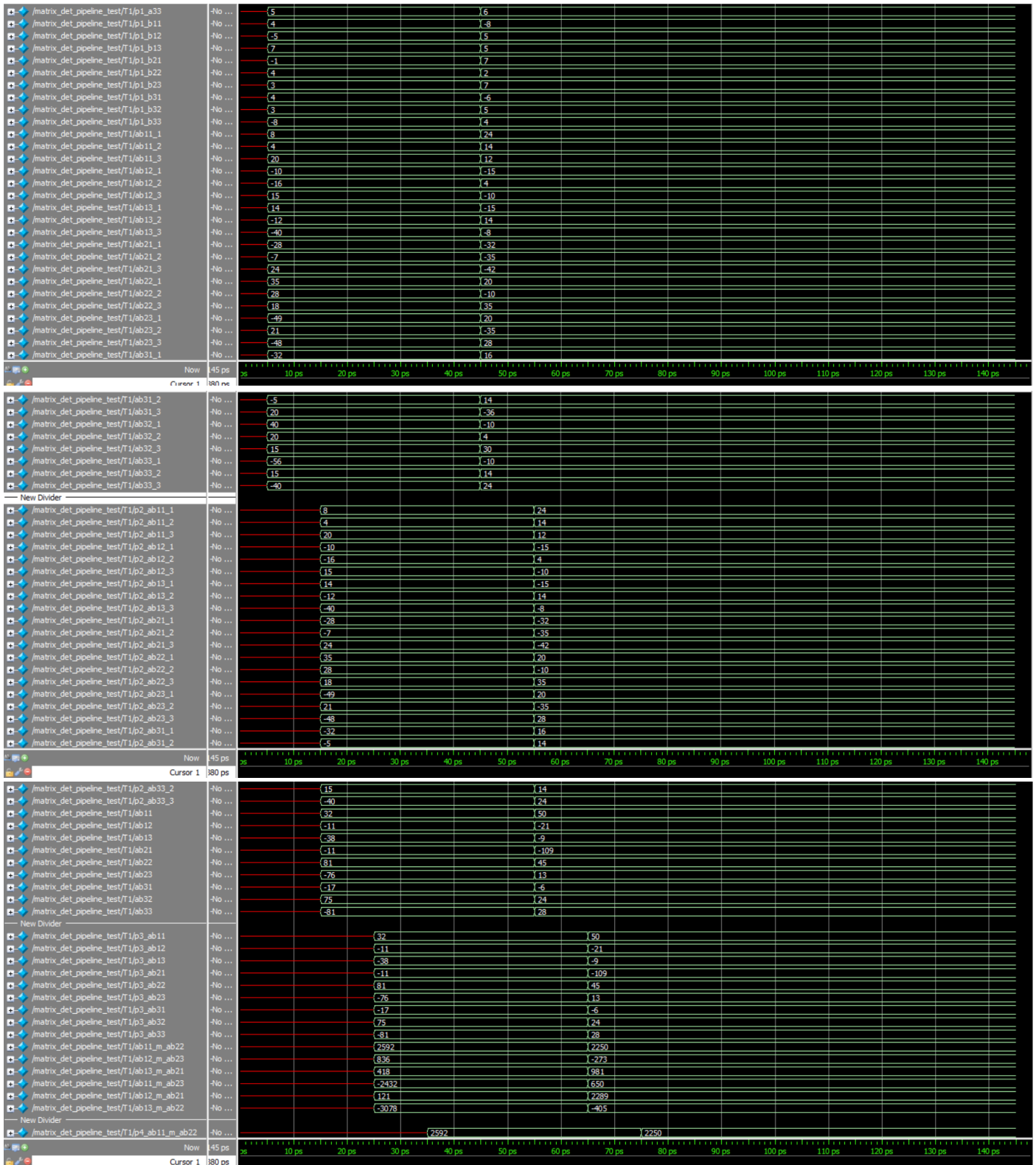
173      a33 = 4'b0101;
174
175      b11 = 4'b0100;
176      b12 = 4'b1011;
177      b13 = 4'b0111;
178      b21 = 4'b1111;
179      b22 = 4'b0100;
180      b23 = 4'b0011;
181      b31 = 4'b0100;
182      b32 = 4'b0011;
183      b33 = 4'b1000;      #45
184      a11 = 4'b1101;
185      a12 = 4'b0010;
186      a13 = 4'b1110;
187      a21 = 4'b0100;
188      a22 = 4'b1011;
189      a23 = 4'b0111;
190      a31 = 4'b1110;
191      a32 = 4'b0010;
192      a33 = 4'b0110;
193
194      b11 = 4'b1000;
195      b12 = 4'b0101;
196      b13 = 4'b0101;
197      b21 = 4'b0111;
198      b22 = 4'b0010;
199      b23 = 4'b0111;
200      b31 = 4'b1010;
201      b32 = 4'b0101;
202      b33 = 4'b0100;
203
204      #100;
205      $stop;
206  end
207  endmodule

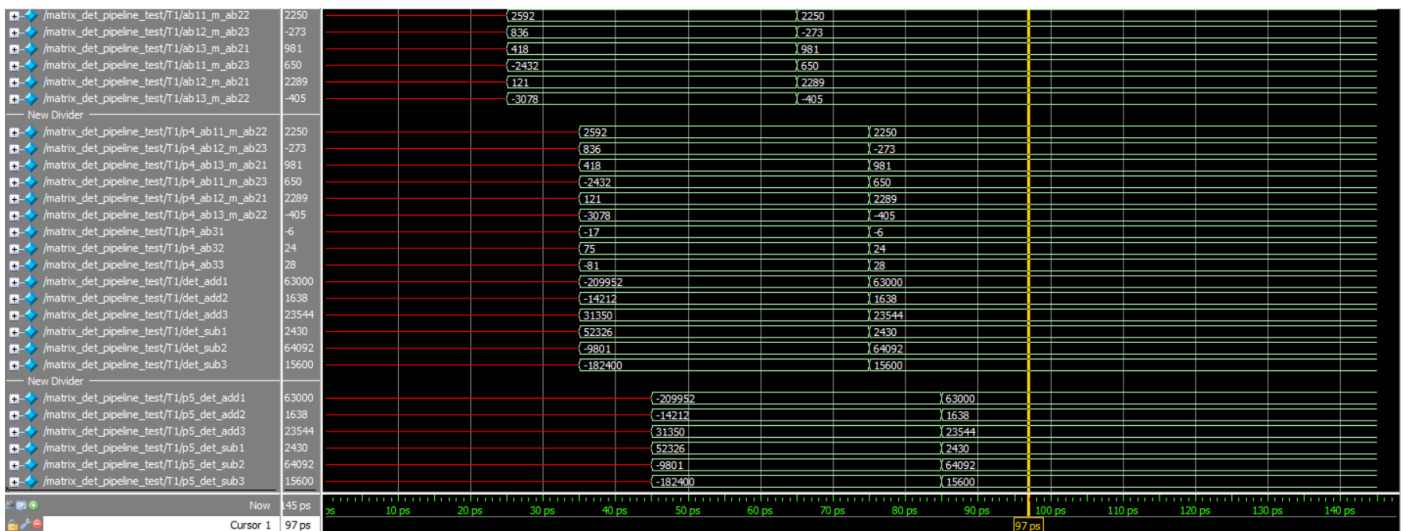
```

波型圖截圖：

//放波型圖截圖







Q2.CN_mult (複數乘法)

Ex: multiplier for 2 complex numbers (1/3)

```

module CN_mult(
  input 8 clk,
  input signed [7:0] a, input signed [7:0] b,
  input signed [7:0] c, input signed [7:0] d,
  output signed [17:0] R_out,
  output signed [17:0] I_out
);
//-----
//--- pipeline of this code ---//
// step1: (a-b), (c-d), (a+b)
// name: a_m_b, c_m_d, a_p_b
// m for minus, p for plus
// step2: a_m_b*d, c_m_d*a, a_p_b*c
// name: A, B, C
// step3: A+B, C-B
// name: R_out, I_out

//--- pipeline structure ---//
// step1 | step2 | step3 (this code)

//--- step1 ---//
wire signed [8:0] a_m_b, c_m_d, a_p_b;

assign a_m_b = a - b;
assign c_m_d = c - d;
assign a_p_b = a + b;

```

```

reg signed [8:0] p1_a_m_b, p1_c_m_d, p1_a_p_b; // DFFs
reg signed [7:0] p1_a, p1_c, p1_d; // DFFs

always @(posedge clk) begin
  p1_a_m_b <= a_m_b; //p1 for pipeline
  p1_c_m_d <= c_m_d;
  p1_a_p_b <= a_p_b;

```

```

  p1_a <= a;
  p1_c <= c;
  p1_d <= d;

end

//--- step2 ---//
wire signed [17:0] A, B, C;

```

```

assign A = p1_a_m_b*p1_d;
assign B = p1_c_m_d*p1_a;
assign C = p1_a_p_b*p1_c;

```

```

reg signed [17:0] p2_A, p2_B, p2_C; // DFFs
always @(posedge clk) begin
  p2_A <= A; //p2 for pipeline 2
  p2_B <= B;
  p2_C <= C;

```

```

end

//--- step3 ---//
assign R_out = p2_A + p2_B;
assign I_out = p2_C - p2_B;
endmodule

```

$$(a+bi)(c+di) = (ac-bd) + (ad+bc)i$$

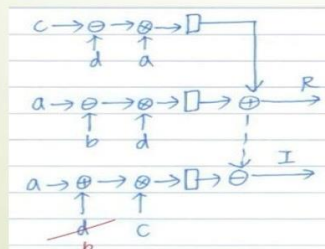
$$A = (a-b)d$$

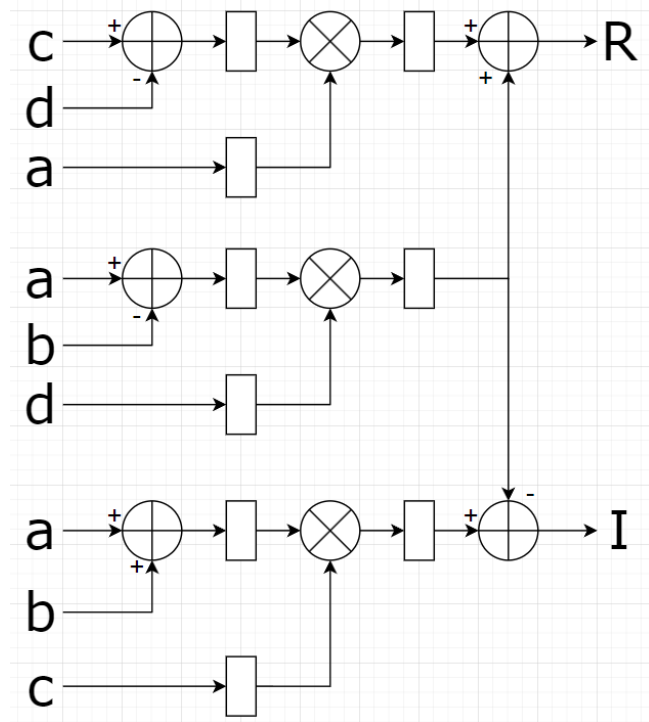
$$B = (c-d)a$$

$$C = (a+b)c$$

$$A+B = ad-bd+ac-ad = ac-bd$$

$$C-B = ac+bc-ac+ad = ad+bc$$





請依據 testbench 的程式片段撰寫測試程式 (Fig. 2-1)，波型圖如下 (Fig. 2-2):

<pre> always #5 clk= ~clk; initial begin clk = 0; a = 8'b1010_1101; b = 8'b0101_0111; c = 8'b1110_1000; d = 8'b0111_1010; #15; a = 8'b0010_1000; b = 8'b0101_0110; c = 8'b1110_1101; d = 8'b1111_1110; </pre>	<pre> #15; a = 8'b1010_1100; b = 8'b0101_0000; c = 8'b1110_1011; d = 8'b0111_1001; #100; \$stop; </pre>
---	--

Fig. 2-1 testbench 的程式片段

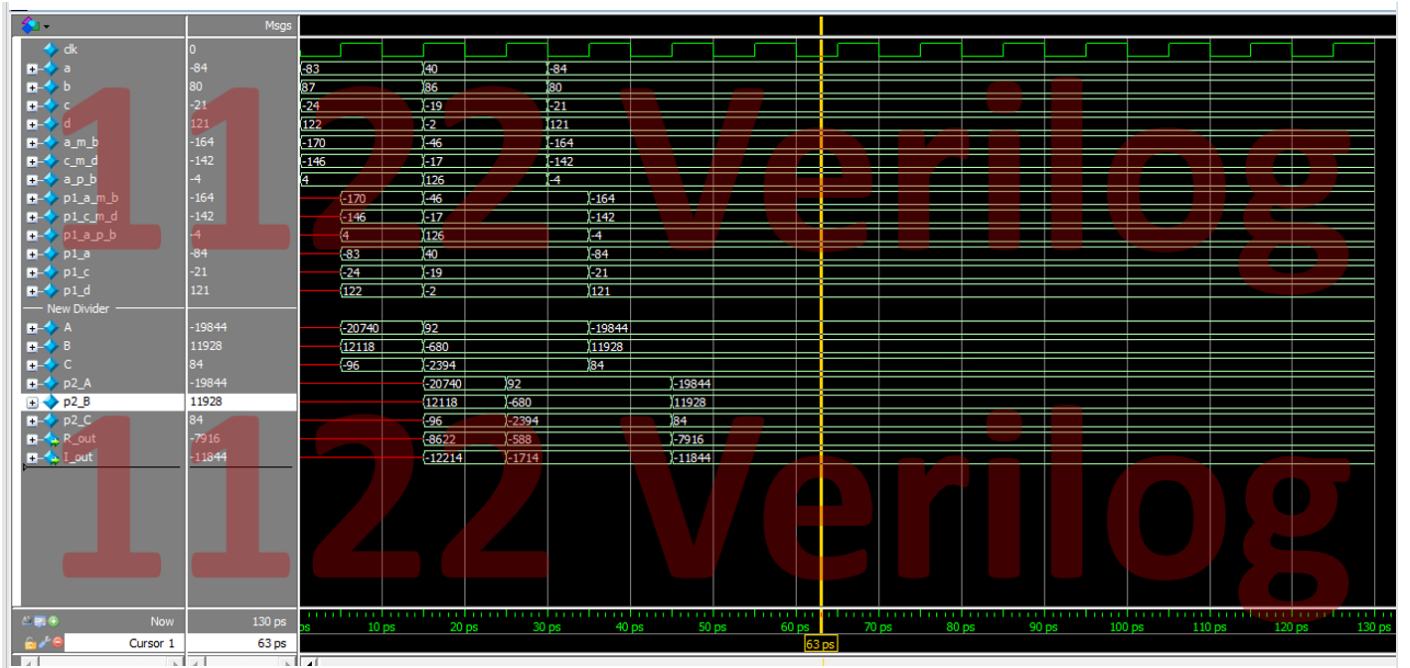


Fig. 2-2 波型圖

CN_mult.v

主程式碼截圖：

//放主程式碼截圖

```

1 module CN_mult(
2     input clk,
3     input signed [7:0] a,b,
4     input signed [7:0] c,d,
5     output signed [17:0] R_out,
6     output signed [17:0] I_out
7 );
8     wire signed [8:0] a_m_b, c_m_d, a_p_b;
9     reg signed [8:0] p1_a_m_b, p1_c_m_d, p1_a_p_b;
10    reg signed [7:0] p1_a, p1_c, p1_d;
11    wire signed [17:0] A, B, C;
12    reg signed [17:0] p2_A, p2_B, p2_C;
13
14    assign c_m_d = c - d;
15    assign a_m_b = a - b;
16    assign a_p_b = a + b;
17
18    always @(posedge clk) //pipeline 1
19    begin
20        p1_a_m_b <= a_m_b;
21        p1_c_m_d <= c_m_d;
22        p1_a_p_b <= a_p_b;
23        p1_a <= a;
24        p1_d <= d;
25        p1_c <= c;
26    end
27
28    assign A = p1_c_m_d*p1_a;
29    assign B = p1_a_m_b*p1_d;
30    assign C = p1_a_p_b*p1_c;
31
32    always @(posedge clk) //pipeline 2
33    begin
34        p2_A <= A;
35        p2_B <= B;
36        p2_C <= C;
37    end
38
39    assign R_out = p2_A + p2_B;
40    assign I_out = p2_C - p2_A;
41 endmodule

```

CN_mult_test.v

測試程式碼截圖：

//放測試程式碼截圖

```

43 module CN_mult_test;
44 reg clk;
45 reg signed [7:0] a,b;
46 reg signed [7:0] c,d;
47 wire signed [17:0] R_out;
48 wire signed [17:0] I_out;
49
50 CN_mult T1(
51     .clk(clk),
52     .a(a),
53     .b(b),
54     .c(c),
55     .d(d),
56     .R_out(R_out),
57     .I_out(I_out)
58 );
59
60 always #5 clk = ~clk;
61
62 initial
63 begin
64     clk = 0;
65     a = 8'b1010_1101;
66     b = 8'b0101_0111;
67     c = 8'b1110_1000;
68     d = 8'b0111_1010;
69
70     #15;
71     a = 8'b0010_1000;
72     b = 8'b0101_0110;
73     c = 8'b1110_1101;
74     d = 8'b1111_1110;
75     #15;
76     a = 8'b1010_1100;
77     b = 8'b0101_0000;
78     c = 8'b1110_1011;
79     d = 8'b0111_1001;
80     #100;
81     $stop;
82 end
83 endmodule

```

波型圖截圖：

//放波型圖截圖

